

核心流程时序图详解

oneworld daemon --remote-access 启动：机器注册 + Relay 连接建立全流程

一、流程概述

该时序图描述了用户启动 `oneworld daemon --remote-access` 后，守护进程如何完成 **本地配置解析** → **用户/组织身份确认** → **机器注册 (upsert)** → **获取 Relay 接入配置** → **建立 WebSocket 长连接并通过 Relay 鉴权** 的完整链路。整体可分为四个阶段：本地准备、身份与注册、获取中继配置、建立中继连接。

参与角色 (Lifelines)

参与者	职责
User	发起命令的终端用户
<code>oneworld daemon --remote-access</code>	本地守护进程，编排整个流程
<code>~/.oneworld/*/computer.json</code>	本地机器配置文件（持久化身份）
<code>Auth / Backend /api/v1/auth/me</code>	认证服务，返回登录用户身份信息
<code>Backend /api/v1/computers</code>	机器注册 (upsert) 接口
<code>Backend /api/v1/computers/:id/relay-config</code>	中继配置下发接口
DaemonCore	守护进程核心，持有 relay 配置并管理连接
<code>Relay /v0/computer/:id/agent</code>	中继服务端，承载 WebSocket Agent 连接

阶段 1 | 本地准备：读取配置与派生临时机器标识

- 1 启动命令：User 执行 `启动 daemon --remote-access`，触发守护进程启动。
- 2 读取本地配置：守护进程调用 `readComputerConfig()`，尝试读取 `~/.oneworld/*/computer.json`。
- alt 分支 [本地没有 computer.json]：若本地无配置，进入首次注册流程：
 - 3. 守护进程向 User 询问是否注册 + **computer name**（交互确认机器名）。
 - 4. 调用 `resolveStableMachineId()`（自调用）：解析稳定的机器硬件标识。
 - 5. 调用 `deriveUnscopedComputerId()`（自调用）：基于机器标识派生出"未绑定组织"的 `computerId`。
 - 6. 保存临时本地配置 到 `computer.json`。

阶段 2 | 身份确认与机器注册

- 7 查询登录身份：守护进程用登录 token 调用 `用登录 token 查 /api/v1/auth/me`，向 Auth 服务请求当前用户信息。
- 8 返回用户信息：Auth 返回 `userId / orgId / defaultBillingTeamId`（用户 ID、组织 ID、默认计费团队）。

9 派生组织级 ID：守护进程调用 `deriveComputerIdForOrg(orgId)`（自调用），基于 `orgId` 生成"组织作用域"的 `computerId`。

alt 分支 [org-scoped id 或 org/team 信息变化]：若组织作用域 ID 或组织/团队信息发生变化，则覆盖保存 `org-scoped computer.json`，使本地配置与组织身份保持一致。

10 注册机器：守护进程调用 `POST /api/v1/computers { computer }`，提交机器信息进行 upsert。

11 服务端校验（自调用）：Backend 执行 校验 `caller` 可 `bill team + org` 匹配，确认调用方有权计费且组织匹配。

12 盖章 `ownerUserId`（自调用）：Backend 执行 `ownerUserId` 从登录用户盖章，以登录用户身份标记机器归属。

13 返回注册结果：Backend 返回 `upserted computer`（已创建或更新的机器记录）。

阶段 3 | 获取中继 (Relay) 配置

14 请求中继配置：守护进程调用 `GET /api/v1/computers/:computerId/relay-config`。

15 归属校验（自调用）：Backend 执行 校验 `caller` 是 `computer owner`，确认调用方为该机器拥有者。

16 下发接入信息：Backend 返回 `relayAgentUrl + 短期 relay token`（中继地址与短时效令牌）。

阶段 4 | 建立中继 WebSocket 连接并鉴权

17 构造核心对象：守护进程执行 `new DaemonCore({ relay config })`，将中继配置交给 `DaemonCore` 管理。

18 建立长连接：DaemonCore 发起 `WebSocket connect /v0/computer/:id/agent`，连接到 Relay。

19 连接鉴权：DaemonCore 发送 `relay.authenticate { token }`，用短期 token 进行鉴权。

20 Relay 校验（自调用）：Relay 执行 校验 `token` 绑定当前 `computerId`，确认 token 与机器匹配。

21 注册连接（自调用）：Relay 执行 `registry.getOrCreate(computerId).setComputer(socket)`，将该 socket 登记到机器注册表。

22 鉴权成功：Relay 返回 `relay.auth_ok`，连接建立完成，机器进入可远程访问状态。

二、关键设计要点

- 双层 `computerId`：先派生 `unscoped`（无组织）ID 用于首次本地落盘，登录后再派生 `org-scoped` ID 并覆盖保存，保证机器身份与组织绑定一致。
- 身份盖章在服务端：`ownerUserId` 由后端从登录用户"盖章"，而非客户端自报，防止越权注册。

- **短期 relay token**：中继令牌是短时效的，且与 computerId 绑定，Relay 端二次校验，降低令牌泄露风险。
- **幂等 upsert**：POST /computers 采用 upsert 语义，重复启动不会产生重复机器记录。
- **连接注册表**：Relay 用 registry 维护 computerId → socket 映射，为后续远程访问转发提供寻址能力。

三、典型时序总结（Happy Path）

启动 → 读本地配置 → （首次则交互注册并派生临时 ID） → /auth/me 取身份 → 派生 org-scoped ID → （变化则覆盖保存） → POST /computers 注册 → GET /relay-config 取中继配置 → new DaemonCore → WebSocket 连接 Relay → authenticate → 校验并登记 → auth_ok ，全链路打通。